A Tutorial-Based Implementation of a Secure Smart Hospital Network Using SDN and Cloud Technologies

Jaden Julius Mascraenhas
Kingston University, London
Master's in information and network security
K2462387

***Abstract*— This tutorial paper presents the design, implementation, and evaluation of a secure smart hospital network that applies Software-Defined Networking (SDN) principles with cloud integration. While a full SDN controller is not implemented, the design demonstrates SDN concepts such as logical segmentation, centralised policy enforcement, and programmable traffic control. Modern healthcare environments increasingly depend on cloud services, interconnected medical devices, and remote access, creating significant security and traffic management challenges. To address these issues, a complete network architecture was designed and evaluated using Cisco Packet Tracer, incorporating VLAN-based segmentation, inter-VLAN routing, a cloud gateway, and access control lists (ACLs) to limit unnecessary communication between hospital network segments. An on-site hospital portal and a cloud-hosted service were deployed to demonstrate secure access to both internal and external resources. Experimental results indicate that the proposed network design effectively isolates traffic, enforces security policies, and maintains reliable cloud connectivity. This paper aims to provide a practical, step-by-step guide for building and testing a secure, cloud-integrated hospital network.**

Keywords—SDN principle, Cloud Networking, VLAN Segmentation, ACL Security, Smart Hospital Network, Packet Tracer (keywords)

I. INTRODUCTION

In today's modern systems, organisations are becoming increasingly dependent on cloud services, connected devices, and remote access, especially in the healthcare sector. Hospitals can now use cloud platforms for storing patient records, running applications, and supporting real-time monitoring of patients [1]. All of this depends on a network that needs to be secure, organised, and capable of managing different kinds of traffic. Implementing the system of least privilege so that users and devices only access the parts of the network they are authorised to is one of the major challenges, particularly when IoT devices, guest users, and hospital staff all share the same infrastructure.

Because of these issues, Software Defined Networking principles offer a new way to tackle these challenges by separating the control and data planes, improving flexibility, and making it easier to apply security rules [2], [3], [6]. VLAN segmentation and access control lists are important because they isolate traffic and reduce the chances of attacks spreading across the network. This is especially useful in hospitals where different departments require different levels of access and security.

This paper focuses on designing, implementing, and testing a secure Smart Hospital network using SDN principles and cloud integration. The network is demonstrated and built using Cisco Packet Tracer, which allows different hospital departments, IoT devices, and guest users to be separated while still providing controlled access to cloud services. This work shows why secure cloud-connected hospital networks are important and explains how SDN, VLANs, and ACLs can be used to improve security in a modern healthcare environment.

II. RELATED WORK & LITERATURE REVIEW

A. SDN in Hospitals

Recent studies have shown that hospitals are migrating to cloud-based infrastructure and Software-Defined Networking (SDN) due to the inability of traditional networking infrastructure to support the growing number of devices and the large amount of data being collected and used in hospitals today. Many studies highlight that hospitals are increasingly employing Electronic Health Records (EHR) systems, cloud storage, telemedicine, and IoT devices, all of which require a versatile and secure network to operate effectively and reliably.

Past approaches to networking infrastructure usually employed flat networking, where most devices are connected within the same broadcast domain. This makes it easier for attacks to spread across the network once a single device is compromised. Because of this issue, researchers have explored SDN and network segmentation as more secure and manageable ways to operate hospital networks.

Several studies discuss how SDN's separation of the control plane and data plane simplifies management and updates. In traditional settings, network devices are often configured manually, which is time-consuming and error-prone. SDN enables centralised control, allowing changes to be made faster and more consistently. This is particularly valuable in hospitals, where departments have different security requirements: IoT medical devices require strict isolation, while administrative staff need controlled access to cloud applications and patient records. Studies indicate that SDN can reduce

Configuration errors and improve security by enabling better control and monitoring of traffic flows in hospital environments.

B. *VLAN Segmentation and ACLs*

The literature emphasises the role of Access Control Lists (ACLs) in enforcing the principle of least privilege. ACLs limit traffic between VLANs and prevent devices from reaching sensitive systems unless explicitly authorised. For example, IoT devices should be prevented from accessing internal hospital servers, while guest users should be restricted to internet access only. ACLs therefore reduce the network attack surface by blocking unnecessary or unauthorised communication paths.

VLANs group devices by function rather than physical location, simplifying segmentation of departments such as emergency, administration, pharmacy, and IoT systems [4]. This logical separation reduces broadcast traffic and limits the spread of malware or misconfigurations. Researchers also note performance improvements from VLAN usage in environments with many connected devices. Together, VLANs and ACLs provide a layered approach that improves security and performance in hospital networks.

C. *Cloud Integration*

Cloud integration is another area that has been widely discussed in the literature. Many papers explain that hospitals are moving to cloud platforms for storing patient data, running applications, and supporting remote access for staff and services. Cloud services offer scalability and reliability, but they also introduce new security challenges that must be addressed.

Researchers highlight the importance of secure gateways, encrypted communication, and proper access control when connecting local hospital networks to cloud services. Some studies also show how SDN can assist in managing cloud traffic by providing better visibility and control over how data moves between the hospital network and the cloud.

D. *Simulation Tools*

Several tutorials and educational papers demonstrate the use of Cisco Packet Tracer for teaching SDN concepts and testing network designs in academic settings. These works show that simulation tools are useful for understanding how VLANs, ACLs, routing, and cloud services work together in a single network environment. Packet Tracer is widely used in academia because it enables realistic network builds without physical hardware [5].

This paper builds on these ideas by combining SDN principles, VLAN segmentation, ACL security, and cloud integration into one practical Smart Hospital network design that can be followed step by step. While existing studies cover these areas individually, few combine them into a single integrated approach for hospital network design. This paper aims to address this gap by presenting an integrated, practical Smart Hospital network model.

III. METHODOLOGY AND PROPOSED WORK

This section describes the process used to design and implement the smart hospital network in Cisco Packet Tracer. This is in a tutorial-oriented approach so the network can be reproduced easily while demonstrating practical concepts like VLAN segmentation, inter-VLAN routing, ACL and cloud:

A. *Building the Network Topology*

The network topology was constructed in Cisco Packet Tracer by placing all required devices and interconnecting them using copper straight-through cables. The topology included a router and an external server used to represent cloud services. This provides a more realistic demonstration of how a hospital network connects to external systems. The complete topology also included a core router, a core switch, an on-premise server, and three end devices (Admin PC, IoT PC, and Guest Laptop).

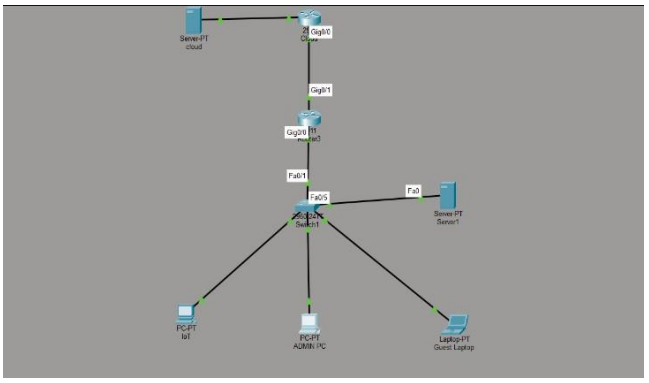

Fig. 1. Smart Hospital Network Topology in Cisco Packet Tracer.

For beginners, a physical connection was made using a simple structure layout. The following physical connections were made:

The core router's GigabitEthernet0/0 interface is connected directly to the core switch on FastEthernet0/1.

The cloud router's GigabitEthernet0/0 interface was linked to the external cloud server on Ethernet1.

End devices were also placed in fixed positions: the Admin PC connected to FastEthernet0/3, the IoT PC to FastEthernet0/2, and the Guest Laptop to FastEthernet0/4 on the core switch.

The on-premise server was connected to FastEthernet0/5 on the switch.

This structured layout ensured that every device was in the correct place for VLAN assignment and later configuration.

The design below separates the network into logical zones, with the core switch acting as the central routing point and the router providing access to external cloud services and an on-premise server.

B. *VLAN Creation*

The next step was to create VLANs on the core switch to segment the hospital network into separate functional zones. VLANs help isolate traffic between different departments and device types, improving both security and performance. In this design, four VLANs were created to represent the IoT devices, administrative systems, guest users, and the on-premise server**.**

To begin creating the VLANs, the user opens the core switch in Cisco Packet Tracer, selects the CLI tab,

The first step is to enter privileged mode by typing enable, which unlocks the switch's advanced commands. From there, configure terminal is used to enter global configuration mode, allowing the VLANs to be created and named.

```
Switch>enable
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#
```

Fig. 2. CLI output showing entry into privileged and global configuration mode on the core switch.

Each VLAN is then created using the vlan command followed by a descriptive name. For example, VLAN 10 and the name IoT create VLAN 10 for IoT devices. This process is repeated for VLANs 20 (Admin), 30 (Guest), and 40 (Server), as shown below.

```
Switch(config)#vlan 10
Switch(config-vlan)#name IOT
Switch(config-vlan)#vlan 20
Switch(config-vlan)#name Admin
Switch(config-vlan)#vlan 30
Switch(config-vlan)#name Guest
Switch(config-vlan)#vlan 40
Switch(config-vlan)#name Server
Switch(config-vlan)#exit
```

Fig. 3. VLAN creation and naming using CLI commands.

After all VLANs are created, the show vlan brief command is used to verify that they exist and are ready for port assignment. This command displays a table showing each VLAN ID, name, and status shown in the image below.

```
Switch#show vlan brief

VLAN Name                             Status    
---- -------------------------------- ---------
1    default                          active    

10   IOT                              active
20   Admin                            active
30   Guest                            active
40   Server                           active
1002 fddi-default                     active
1003 token-ring-default               active
1004 fddinet-default                  active
1005 trnet-default                    active
```

Fig. 4. VLAN summary output using show vlan brief.

This completes the VLAN setup and prepares the switch for assigning ports to each VLAN in the next step.

C. *Assigning Ports to VLANs*

After creating the VLAN, the next step is to assign specific switch ports to each VLAN. This ensures that each device is in the correct logical segment of the network. Port assignment is done using simple CLI commands on the switch, where the process begins by selecting a physical port using the interface command.

For example, interface fa0/2 selects FastEthernet port 0/2. Then, switchport mode access is used to set the port to access mode, which means it will carry traffic for only one VLAN. Finally, the switchport access vlan command assigns the port to a specific VLAN.

The table below shows how each device was assigned to its corresponding VLAN:

Device	Switch Port	Assigned VLAN
IoT PC	Fa0/2	VLAN 10
Admin PC	Fa0/3	VLAN 20
Guest Laptop	Fa0/4	VLAN 30
On-premise Server	Fa0/5	VLAN 40

Table 1. Device-to-VLAN Port Assignments

After confirming the device-to-VLAN mapping in Table 1, the switch ports were configured accordingly using the CLI. Each port was placed into access mode and assigned to its designated VLAN using the interface, switchport mode access, and switchport access vlan commands. This ensures that every device is correctly segmented within the network. The full CLI output for the port-assignment process is shown in Fig. 5 below.

```
Switch(config)#interface fa0/2
Switch(config-if)# switchport mode access
Switch(config-if)# switchport access vlan 10
Switch(config-if)#exit
Switch(config)#
Switch(config)#interface fa0/3
Switch(config-if)# switchport mode access
Switch(config-if)# switchport access vlan 20
Switch(config-if)#exit
Switch(config)#
Switch(config)#interface fa0/4
Switch(config-if)# switchport mode access
Switch(config-if)# switchport access vlan 30
Switch(config-if)#exit
Switch(config)#
Switch(config)#interface fa0/5
Switch(config-if)# switchport mode access
Switch(config-if)# switchport access vlan 40
```

FIG. 5. CLI OUTPUT SHOWING SWITCH PORT ASSIGNMENTS TO THEIR RESPECTIVE VLANS.

D. *Configuring the Trunk Link*

The next step is to configure the trunk link between the core switch and the router. The trunk allows all VLANs to travel over a single physical connection, enabling inter-VLAN routing. The switch port connected to the router is selected using the interface command, then configured with switchport mode trunk to enable trunking. The command switchport trunk allowed vlan all is applied, so VLANs 10, 20, 30, and 40 can pass through the link.

```
Switch(config)#interface fa0/1
Switch(config-if)# switchport mode trunk
Switch(config-if)# switchport trunk allowed vlan 10,20,30,40
Switch(config-if)#exit
```

Fig. 6. CLI output showing trunk configuration on the core switch.

E. Configuring Router Sub-Interfaces (Inter-VLAN Routing)

To enable communication between VLANs, sub-interfaces were created on the router's GigabitEthernet0/0 interface. Each sub-interface was linked to a VLAN using the encapsulation dot1Q command and assigned an IP address to serve as the default gateway for that VLAN. Sub-interfaces were created for VLANs 10, 20, 30, and 40. Completing the inter-VLAN routing setup.

The CLI output for this configuration is shown in Fig. 7 below.

```
Router(config)#interface g0/0.10
Router(config-subif)# encapsulation dot1Q 10
Router(config-subif)# ip address 192.168.10.1 255.255.255.0
Router(config-subif)#exit
Router(config)#
Router(config)#interface g0/0.20
Router(config-subif)# encapsulation dot1Q 20
Router(config-subif)# ip address 192.168.20.1 255.255.255.0
Router(config-subif)#exit
Router(config)#
Router(config)#interface g0/0.30
Router(config-subif)# encapsulation dot1Q 30
Router(config-subif)# ip address 192.168.30.1 255.255.255.0
Router(config-subif)#exit
Router(config)#
Router(config)#interface g0/0.40
Router(config-subif)# encapsulation dot1Q 40
Router(config-subif)# ip address 192.168.40.1 255.255.255.0
Router(config-subif)#exit
```

Fig. 7. Router sub-interface configuration for inter-VLAN routing.

F. Assigning IP Addresses to End Devices

After configuring inter-VLAN routing on the router, each end device was assigned a static IP address from its respective VLAN subnet. This ensures that every device can communicate within its VLAN and reach the router's gateway for inter-VLAN communication. IP settings were configured manually on each device through the Desktop → IP Configuration menu in Cisco Packet Tracer**.**

Device	VLAN	IP Address	Subnet Mask	Default Gateway
IoT PC	**10**	**192.168.10.10**	**255.255.255.0**	**192.168.10.1**
Admin PC	**20**	**192.168.20.10**	**255.255.255.0**	**192.168.20.1**
Guest Laptop	**30**	**192.168.30.10**	**255.255.255.0**	**192.168.30.1**
On-premise Server	**40**	**192.168.40.10**	**255.255.255.0**	**192.168.40.1**
Hospital Router3 (Hospital → Cloud link)	**—**	**200.200.200.1**	**255.255.255.0**	**—**
Cloud	**—**	**200.200.200.2**	**255.255.255.0**	**—**
Router				
Cloud Server	**—**	**200.200.201.3**	**255.255.255.0**	**200.200.201.1**

The assigned IP addresses for all devices are shown in Table 2.

G. Configuring ACLs (Access Control Lists)

To enforce security between the VLANs, Access Control Lists (ACLs) were applied on the router. ACLs allow the network to control which devices or VLANs are permitted to communicate with others. In this design, ACLs were used to restrict guest users, protect the on-premise server, and ensure that only authorised devices can access sensitive resources. Each ACL was created in global configuration mode and then applied to the appropriate sub-interface in the inbound direction.

The CLI output for the ACL configuration is shown.

```
Router(config)#access-list 100 deny ip 192.168.30.0 0.0.0.255 192.168.20.0 0.0.0.255
Router(config)#access-list 100 permit ip any any
Router(config)#
Router(config)#interface g0/0.30
Router(config-subif)# ip access-group 100 in
Router(config-subif)#exit
Router(config)#
Router(config)#! Allow Admin VLAN (20) to access Server VLAN (40)
Router(config)#access-list 110 permit ip 192.168.20.0 0.0.0.255 192.168.40.0 0.0.0.255
Router(config)#access-list 110 deny ip any any
Router(config)#
Router(config)#interface g0/0.20
Router(config-subif)# ip access-group 110 in
Router(config-subif)#exit
```

Fig. 8. ACL configuration applied to router sub-interfaces.

H. Test Procedure and Verification Methodology

Network testing is performed using the ping command from any end device in the command prompt option in the desktop tab, for example, the Guest Laptop. The device successfully reached another host within its own VLAN (192.168.30.10). To verify cloud connectivity, the Admin PC (VLAN 20) was used to ping the external cloud server (IP: 200.200.201.3). The successful replies confirm that authorised devices can securely access external cloud services while maintaining VLAN isolation and ACL restrictions. Confirming local connectivity. Attempts to ping devices in other VLANs — including the Admin PC (192.168.20.10) and IoT PC (192.168.10.10) — returned "Destination host unreachable," verifying that ACLs were correctly blocking inter-VLAN access. These results confirmed that the network was segmented and secure as intended.

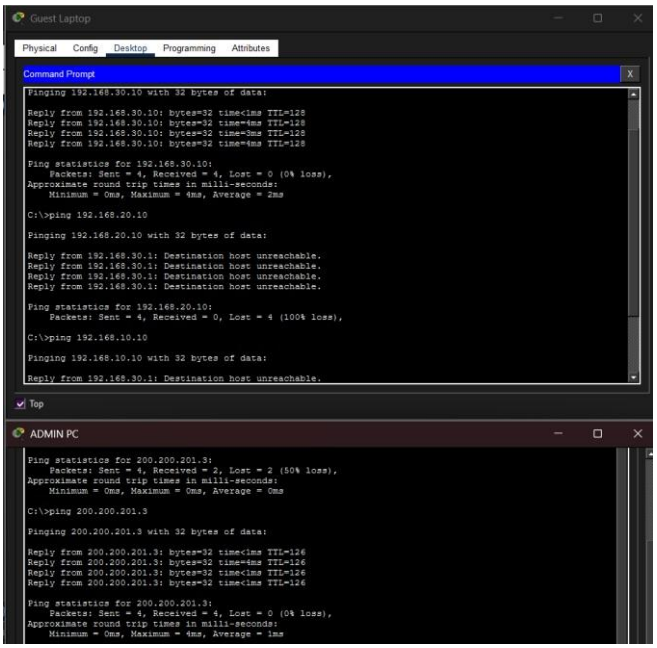

Fig. 9. Ping results from Guest Laptop and Admin PC demonstrating VLAN isolation, ACL enforcement, and successful cloud connectivity.

IV. Experimental Setup

The experimental setup was implemented using Cisco Packet Tracer to provide a controlled and repeatable environment for validating VLAN segmentation, inter-VLAN routing, and ACL-based access control. The simulation workspace was arranged to reflect the final network design, with all devices placed in their operational positions, powered on, and connected through the configured VLANs, trunk links, and router sub-interfaces.

To ensure consistency during testing, all devices were configured, including IP addressing, VLAN assignments, ACL rules, and routing settings, which were then saved and verified. The Packet Tracer environment allowed real-time observation of link status, interface activity, and protocol behaviour, enabling accurate validation of connectivity and security policies. The cloud router and external server were included to emulate remote healthcare services, while the on-premise server and end devices represented internal hospital systems. While Packet Tracer does not fully emulate real SDN controllers or production traffic loads, it effectively validates the logical segmentation, routing, and access control being implemented for educational and design evaluation purposes [6].

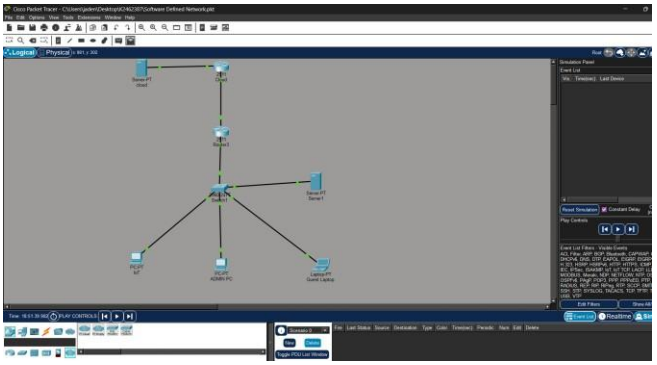

Fig. 10. Cisco Packet Tracer simulation workspace illustrating the experimental environment used for network testing.

V. Results and Discussion

Cisco Packet Tracer was used to test the deployed smart hospital network to verify VLAN segmentation, inter-VLAN routing, and ACL enforcement. Proper segmentation was confirmed by each device's successful communication within its designated VLAN (see Fig. 11).

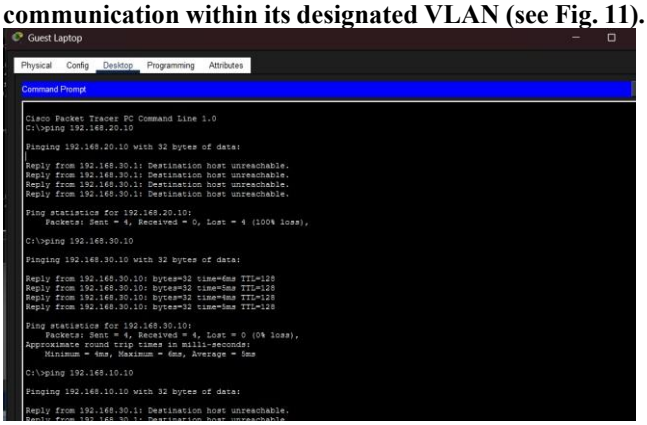

Fig. 11. Ping results showing VLAN segmentation and ACL enforcement.

In the simulation, ACLs properly enforced isolation: the Guest Laptop (VLAN 30) successfully pinged another host in VLAN 30 with 0% packet loss, demonstrating correct intra-VLAN connectivity. ACL enforcement was validated by attempting cross-VLAN communication: the Guest Laptop could not reach the Admin PC (VLAN 20) or IoT devices (VLAN 10), with attempts returning "Destination host unreachable," confirming that ACLs block unauthorised inter-VLAN traffic (see Fig. 11). Permitted traffic was allowed: the Admin PC (VLAN 20) successfully reached both the on-premise server (VLAN 40) and the external cloud server, demonstrating controlled inter-VLAN routing and authorised cloud access (see Fig. 12)

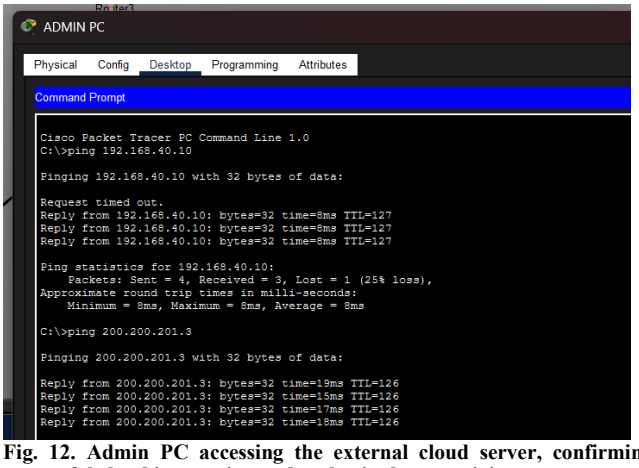

Fig. 12. Admin PC accessing the external cloud server, confirming successful cloud integration and authorised connectivity

Measured round-trip times for successful pings were low (average ≈ 1–4 ms), and packet loss for authorised cloud pings was 0% after initial transient loss. The network's ability to safely integrate with cloud services was confirmed by using authorised devices to access the external server. These findings demonstrate how ACLs and VLAN segmentation implement the principle of least privilege, blocking unwanted access while permitting authorised traffic.

This testing shows that the network design allows secure, controlled communication between hospital departments and cloud services. While Packet Tracer cannot fully replicate production SDN controllers or heavy traffic loads, it is sufficient to validate logical segmentation, routing, and

ACLs;*real-world deployment should include testing with an SDN controller and live traffic to confirm performance and security.

VI. Conclusion and Future Work

The tutorial demonstrated a practical design and Packet Tracer implementation of a secure, cloud-integrated smart hospital network using SDN principles, VLAN segmentation, inter-VLAN routing, and ACLs. Experimental tests confirmed correct intra-VLAN communication, effective ACL enforcement to block unauthorised cross-VLAN traffic, and

authorised access to on-premise and external cloud services. These final results show the proposed architecture enforces least-privilege segmentation while enabling controlled cloud connectivity.

Future work

- Validate the design with a production SDN controller (e.g., OpenDaylight) to assess dynamic policy enforcement and scalability.
- Extend security by adding encrypted cloud links (VPN/TLS) and IoT-specific device authentication, then evaluate performance under realistic traffic

References

[1] W. Stallings, Foundations of Modern Networking: SDN, NFV, QoE, IoT, and Cloud, Pearson, 2020.

[2] N. McKeown et al., "OpenFlow: Enabling innovation in campus networks," ACM SIGCOMM Computer Communication Review, vol. 38, no. 2, pp. 69–74, 2008.

[3] D. Kreutz, F. Ramos, P. Verissimo, C. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," Proceedings of the IEEE, vol. 103, no. 1, pp. 14–76, 2015.

[4] Cisco Systems, "VLAN Configuration Guide," Cisco Documentation, 2023.

[5] Cisco Networking Academy, "Cisco Packet Tracer – Simulation Tool Overview," Cisco Systems, 2023.

[6] T. Nadeau and K. Grey,

SDN: Software Defined Networks, O'Reilly Media, 2013.